# Database Automation Pro

## Technical Report

Enterprise database administration automation platform for PostgreSQL, MySQL, MongoDB, Microsoft SQL Server and Oracle

| | |
|---|---|
| Project | Database Automation Pro (DB AutoPro / Data Automation Pro) |
| Package version | 1.0.0 (pyproject.toml); installer demo build 0.0.1-demo |
| Repository | TechBerryAI / DB-AutoPro - branch ‘Ganesh’ (main branch: production) |
| Report date | 30 October 2026 |
| Prepared by | DB AutoPro Team |
| License | MIT |

*This report was produced by analysing the repository source tree (api/, backend/, desktop/, config/, deployment/, tests/, docs/). Limitations described reflect what the code does today.*

# Table of Contents

**1. Executive Summary** 4

**2. Project Objectives and Scope** 4

Objectives . . . 4

Intended users . . . 4

Out of scope (current version) . . . 5

**3. System Requirements** 5

3.1 Functional requirements . . . 5

3.2 Non-functional requirements . . . 5

3.3 Hardware and software requirements . . . 6

3.4 External tools (must be installed on the machine running the job, or on the remote Linux host) . . . 6

**4. Technology Stack and Dependencies** 6

**5. System Architecture** 7

5.1 Design principles . . . 7

5.2 Request flow (backup via API) . . . 7

5.3 Application lifecycle . . . 8

**6. Project Structure** 8

**7. Module Descriptions** 8

7.1 Connectors . . . 8

7.2 Backup service . . . 8

7.3 Restore service . . . 9

7.4 Patch management . . . 9

7.5 FDW service . . . 9

7.6 Monitoring and reporting . . . 9

7.7 Scheduler . . . 9

7.8 SSH subsystem . . . 9

**8. Migration Framework** 9

**9. REST API** 10

9.1 Role-permission matrix . . . 11

**10. Data Model** 11

**11. Desktop Application** 12

**12. Security Design** 12

Hardening required before production . . . 12

**13. Configuration** 13

**14. Installation and Build Guide** **13**

14.1 Development setup . . . . . 13

14.2 Running . . . . . 13

14.3 Production API deployment . . . . . 13

14.4 Windows installer . . . . . 14

14.5 Troubleshooting . . . . . 14

**15. Testing and Quality Assurance** **14**

**16. Limitations, Risks and Roadmap** **15**

Current limitations . . . . . 15

Suggested roadmap . . . . . 15

**17. Conclusion** **15**

**Appendix A - Quick reference commands** **15**

# 1. Executive Summary

Database Automation Pro is a Python platform that gives database administrators one console, and an optional REST API, for the routine work of running heterogeneous database estates: connecting to servers (locally or through SSH), taking and restoring backups with each engine's native tooling, watching health and performance, applying reviewed SQL and package patches, migrating data between engines, scheduling jobs, and exporting auditable reports.

The product has two front ends over a shared service layer: a Windows desktop application built with CustomTkinter, and a FastAPI service with JWT authentication and role-based access control. Five engines are supported - PostgreSQL, MySQL, MongoDB, Microsoft SQL Server and Oracle - behind a common connector interface.

| Metric | Value |
|---|---|
| Python source files (api / backend / desktop) | 14 / 93 / 67 |
| Approx. lines of Python (incl. tests) | ~52,800 |
| Automated test modules | 40 (pytest) |
| REST endpoints | 26 under /api/v1 plus / and /health |
| Supported engines | PostgreSQL, MySQL, MongoDB, SQL Server, Oracle |
| Migration pairs implemented | 9 (PostgreSQL to/from other engines, plus PG to PG) |
| Target platform | Windows 10/11 desktop; API on any OS with Python 3.10+ |

# 2. Project Objectives and Scope

## Objectives

- Provide a single console to connect to PostgreSQL, MySQL, MongoDB, SQL Server and Oracle, locally or over SSH.
- Run backup and restore using engine-native tools (pg_dump/pg_restore, mysqldump, mongodump/mongorestore, T-SQL BACKUP/RESTORE, Data Pump).
- Surface live health and performance: connections, latency, host CPU/RAM/disk, and audit-based alerts.
- Support SQL patch execution, MySQL package patch checking, PostgreSQL FDW setup, schema/data migration and scheduled jobs.
- Produce reports and execution logs exportable to CSV and PDF, with an audit trail of who did what.
- Package the desktop client as a Windows installer for demos and field use.

## Intended users

| Role | Responsibilities |
|---|---|
| Admin | Full access: connections, backup/restore, patch rollback, scheduler, user management |
| Operator | Day-to-day backup, restore, patching, migration, monitoring and reports |
| Viewer | Read-only monitoring, backup/restore status and reports |

### Out of scope (current version)

- MySQL package *installation* - the patch workflow runs prechecks and an `apt-get -s` dry-run only.
- Packaging the FastAPI service and web dashboard in the Windows installer (desktop only).
- Bundling database client tools (pg_dump, mysqldump, mongodump ...); they must be on PATH.
- Migration of views, functions, triggers and other programmable objects (reported, not copied).

# 3. System Requirements

## 3.1 Functional requirements

| ID | Requirement | Implemented by |
|---|---|---|
| FR-01 | Save, test and manage connection profiles; optional SSH tunnel to Linux hosts | desktop/connection*.py, ssh_tunnel.py, connectors/ |
| FR-02 | Browse databases / server inventory | desktop/explorer/, views/databases_view.py |
| FR-03 | Logical and physical backups; full, incremental, differential modes; compression; retention | backup_service.py, backup_mode_handlers.py, backup_chain.py |
| FR-04 | Restore from local or remote artifacts with validation | restore_service.py, mongo_restore.py, restore_catalog.py |
| FR-05 | Live dashboard: KPIs, host gauges, DB performance, security/activity panels | views/dashboard.py, db_performance.py, monitoring_service.py |
| FR-06 | Execute reviewed SQL patch scripts with history and rollback | patch_service.py, postgres_patch/ |
| FR-07 | MySQL package patch check, plan, approval, dry-run (Ubuntu APT / CentOS DNF adapters) | mysql_patch/ |
| FR-08 | PostgreSQL FDW setup and validation | fdw_service.py, api/routes/fdw.py |
| FR-09 | Schema/data migration between supported engine pairs with precheck, progress, cancel, reconcile | pg_migration/, pg_mysql_migration/, to_pg_migration/ ... |
| FR-10 | Cron-style scheduled backups and retention cleanup | scheduler_service.py (APScheduler) |
| FR-11 | Operations report and execution log with filter, search, CSV/PDF export | report_service.py, desktop/report_export.py, log_export.py |
| FR-12 | JWT login and role-based authorisation on REST API | api/auth/, api/routes/ |
| FR-13 | Light/dark theme and persisted UI preferences | desktop/theme.py, settings_view.py |

## 3.2 Non-functional requirements

| Area | Requirement / design response |
|---|---|
| Security | JWT with expiry; bcrypt hashing; RBAC; pinned SSH host keys; Windows DPAPI for stored secrets; log/redaction of secrets |
| Reliability | Migration operations persisted in SQLite with crash reconciliation; backup validation; disposable-restore proof |
| Auditability | Structured JSON logging, separate audit.log, patch and migration history |
| Safety | Dry-run first, explicit approvals with expiry (30 min), parameterised inserts, locks against concurrent jobs |
| Portability | Config via YAML profiles and DAP_ environment overrides; runtime state isolated under var/ |

| Area | Requirement / design response |
|---|---|
| Extensibility | New engine = new BaseConnector subclass registered in factory; explicit migration capability matrix |
| Testability | 40 pytest modules using fakes for SSH/CLI; live integration tests opt-in via environment variables |

### 3.3 Hardware and software requirements

| Item | Minimum | Recommended |
|---|---|---|
| Operating system | Windows 10 x64 (desktop); Linux/macOS (API only) | Windows 11 x64 |
| Python | 3.10 | 3.12 (used by CI) |
| CPU / RAM | 2 cores / 4 GB | 4 cores / 8 GB |
| Disk | 500 MB for app + free space for backups (sized to databases) | SSD; separate volume for backups |
| Display | 1280x720 | 1920x1080 |
| Network | Reach to DB ports (5432, 3306, 27017, 1433, 1521) and SSH 22 | VPN / private network |

### 3.4 External tools (must be installed on the machine running the job, or on the remote Linux host)

| Engine | Tools required |
|---|---|
| PostgreSQL | pg_dump, pg_restore, psql, pg_basebackup (PostgreSQL client tools) |
| MySQL | mysqldump, mysql (MySQL client tools) |
| MongoDB | mongodump, mongorestore (MongoDB Database Tools; located by mongo_tools.py) |
| SQL Server | ODBC Driver 17+ for SQL Server (pyodbc); T-SQL BACKUP / RESTORE |
| Oracle | Instant Client / Data Pump (expdp, impdp) where wired |
| Build only | PyInstaller; Inno Setup 6 (winget install JRSoftware.InnoSetup) |

## 4. Technology Stack and Dependencies

All runtime dependencies are pinned as minimums in `requirements.txt`. Install with `pip install -r requirements.txt`.

| Category | Library (min. version) | Purpose |
|---|---|---|
| API | fastapi 0.109, uvicorn[standard] 0.27, python-multipart 0.0.6 | REST API, ASGI server, uploads |
| Auth | python-jose[cryptography] 3.3, passlib[bcrypt] 1.7.4, bcrypt >=4.0 <5.0 | JWT, password hashing |
| Config | pydantic 2.5, pydantic-settings 2.1, pyyaml 6.0.1, python-dotenv 1.0 | Models, settings, YAML, .env |
| DB drivers | psycopg2-binary 2.9.9, mysql-connector-python 8.2, pymongo 4.6.1, pyodbc 5.0.1, oracledb 2.0 | Engine connectivity |
| Scheduling / metrics | apscheduler 3.10.4, psutil 5.9.7 | Cron jobs, host CPU/RAM/disk |

| Category | Library (min. version) | Purpose |
|---|---|---|
| Desktop UI | customtkinter 5.2.1, Pillow 10.0, resvg_py 0.3, fpdf2 2.7.9 | GUI, images, SVG icons, PDF export |
| SSH | paramiko 3.4 | Tunnels and remote commands |
| Testing | pytest 7.4.4, pytest-asyncio 0.23.3, httpx 0.26 | Unit / API tests |
| Utilities | email-validator 2.1 | Schema validation |

Packaging tools: PyInstaller (spec: deployment/DataAutomationPro.spec) and Inno Setup (deployment/installer/DataAutomationPro.iss). Lint configuration (ruff, line length 100, py310 target) is in pyproject.toml.

# 5. System Architecture

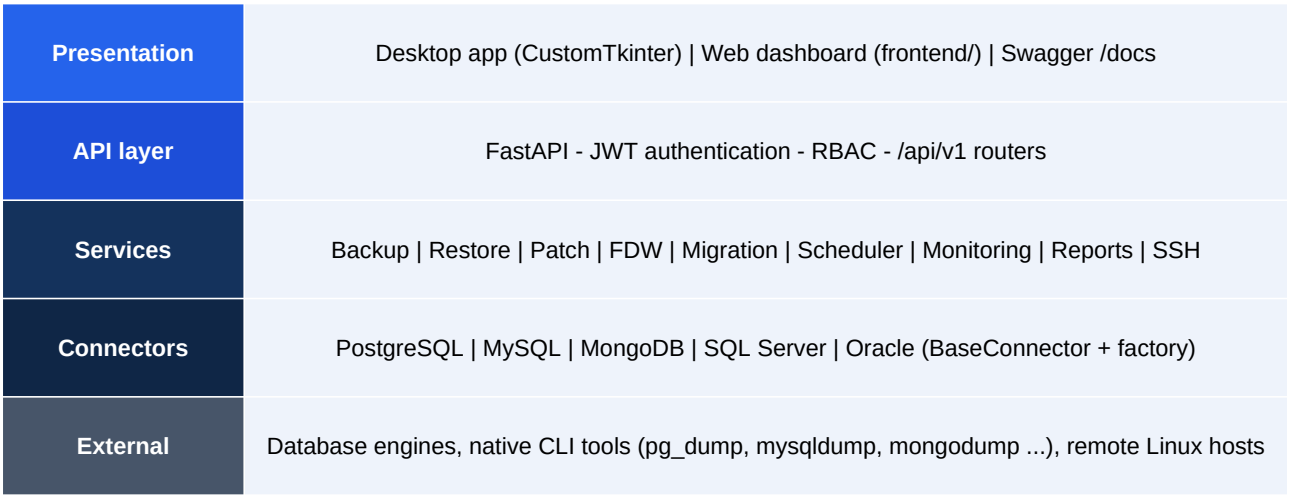



*Figure 1 - Layered architecture. The desktop client calls services directly; the REST API exposes the same services over HTTP.*

The architecture is layered. The **desktop** application invokes backend services directly, so the REST API is not required for day-to-day UI work. The **API** exposes the same capabilities for automation and integration. **Services** own workflows (backup chains, catalogs, audit, patches, migrations); **connectors** hide engine differences.

## 5.1 Design principles

| Principle | Realisation |
|---|---|
| Single responsibility | One service per domain (backup, restore, patch, FDW, migration ...) |
| Open / closed | New engines added through new connector classes and registry entries |
| Liskov substitution | All connectors implement BaseConnector (test_connection, execute_query, execute_script, get_version, health_check) |
| Dependency inversion | Services obtain connectors via get_connector() / get_connector_from_info() |

## 5.2 Request flow (backup via API)

```
User -> POST /api/v1/backups/
  API: validate JWT, check RBAC permission 'backup:create'
  BackupService.create_backup(): validate mode -> resolve output path
     -> engine handler (pg_dump | mysqldump | mongodump | T-SQL | Data Pump), local or via SSH
```

```
    -> compress (gzip) -> record chain/catalog -> audit log
<- OperationResult (status, path, size, timings)
```

### 5.3 Application lifecycle

On API start-up (`api/main.py`) the lifespan handler configures logging, calls `MigrationOperationStore.reconcile_interrupted()` to mark migrations left running by a crashed process, and starts the APScheduler background scheduler; it is stopped on shutdown. CORS is enabled for all origins (development default - restrict in production).

## 6. Project Structure

| Path | Contents |
|---|---|
| api/ | FastAPI app (main.py), auth (jwt_handler, rbac), dependencies, routes: auth, backup, restore, patch, fdw, migration, monitoring |
| backend/core/ | config (YAML + env settings), logging, exceptions, security (secrets, masking), paths |
| backend/models/ | Pydantic schemas and enums (schemas.py) |
| backend/connectors/ | base, postgresql, mysql, mongodb, mssql, oracle |
| backend/services/ | backup/restore/patch/fdw/migration/scheduler/monitoring/report/ssh services; sub-packages mysql_patch, postgres_patch, pg_migration, pg_mysql_migration, pg_sql_migration, pg_mongo_migration, to_pg_migration, migration_framework |
| desktop/ | CustomTkinter app: app.py, views/, explorer/, theme, design system, dialogs, connection store, DPAPI, SSH tunnel and host-trust UI |
| config/ | default.yaml, dev.yaml, test.yaml, prod.yaml, ui_preferences.json |
| frontend/ | Optional lightweight web dashboard (index.html, css, js) |
| deployment/ | PyInstaller spec, Inno Setup script, build.bat, version.txt, assets |
| docs/ | architecture, api, installation, mysql_patch_management, ui_inventory, ui_plan |
| scripts/ | ML dataset export, connection-panel builder |
| tests/ | 40 pytest modules |
| var/ | Runtime state: data/, logs/, backups/ (git-ignored contents) |
| run.py / run_app.py | Entry points: API or desktop; GUI-only launcher used by PyInstaller |
| .github/workflows/ci.yml | CI: Windows, Python 3.12, pytest on push/PR |

## 7. Module Descriptions

### 7.1 Connectors

`BaseConnector` (ABC) takes a `ConnectionInfo` and defines `test_connection`, `execute_query`, `execute_script`, `get_version` and `health_check`. `resolve_connection(profile)` reads connection settings from configuration; `get_connector(db_type, profile)` is the factory.

### 7.2 Backup service

- Engines: PostgreSQL (pg_dump / pg_dumpall / pg_basebackup), MySQL (mysqldump), MongoDB (local, via SSH, or remote endpoint), SQL Server (T-SQL BACKUP), Oracle (Data Pump).

- Categories logical/physical; modes full, incremental, differential - handled by backup_mode_handlers.py with chain metadata in backup_chain.py.
- Scopes and formats (PgBackupScope, PgDumpFormat, MySQLBackupScope, MySQLDumpFormat) allow database, schema or table level dumps.
- gzip compression, retention policy (apply_retention_policy), local and remote backup listing, and backup validation.

### 7.3 Restore service

Restores from local or remote sources to local or remote targets (RestoreSourceType / RestoreTargetType), with validation. MongoDB restores are planned by `mongo_restore.py` (scope: MongoRestoreScope) and tools are located by `mongo_tools.py`. `restore_catalog.py` describes options to the UI.

### 7.4 Patch management

- **SQL script patching** (patch_service.py, postgres_patch/): executes reviewed SQL, keeps history in var/logs/patch_history.json, supports rollback (admin only). PostgreSQL workflow includes read-only administrator pre-checks (admin_check.py), identity checks and a script runner.
- **MySQL package patching** (mysql_patch/): discovery, precheck (1 GiB free disk on /, /var and APT cache; SSH host-key trust), preview of package changes, plan with digest, approval (30-minute expiry), dry-run executor and verification. Adapters: Ubuntu APT and CentOS DNF. Jobs are audited to var/data/mysql_patch_jobs.json with secrets redacted.
- The workflow refuses: non-Ubuntu/non-APT hosts, third-party repositories, series changes (e.g. 8.0 to 8.4), downgrades, removals, and concurrent jobs. This version never installs packages.

### 7.5 FDW service

Creates the postgres_fdw extension, foreign servers, user mappings and foreign tables, and validates configuration (POST /fdw/setup, /fdw/extension, /fdw/validate).

### 7.6 Monitoring and reporting

monitoring_service.py aggregates health checks, dashboard statistics, system metrics (psutil), backup reports and audit-log entries. report_service.py builds the enterprise report model; the desktop exports CSV/PDF (fpdf2). ml_dataset_export.py and scripts/export_ml_dataset.py create a unified CSV dataset from logs, audit, health and metrics for ML use.

### 7.7 Scheduler

SchedulerService wraps APScheduler's BackgroundScheduler with CronTrigger jobs for scheduled backups (default cron 0 2 * * *) and retention cleanup.

### 7.8 SSH subsystem

Paramiko-based client (ssh_client.py), tunnel helpers (desktop/ssh_tunnel.py) and pinned host keys (ssh_host_keys.py): the first connection stores the presented key; a later different key is rejected and never silently replaced. desktop/remote_directory_browser.py lets users pick remote backup directories.

# 8. Migration Framework

Migration is organised around an explicit **compatibility matrix** (migration_framework/registry.py). A source/target pair is offered in the UI only when a connector, inspection path, writer, type mapping and verification path exist. Each pair is described by a PairCapability record (schema, data, type conversion, constraints, indexes, views, programmable objects, verification, limitations).

| Source | Target | Engine module |
|---|---|---|
| PostgreSQL | PostgreSQL | pg_migration |
| PostgreSQL | MySQL | pg_mysql_migration |
| PostgreSQL | SQL Server | pg_sql_migration |
| PostgreSQL | Oracle | pg_sql_migration |
| PostgreSQL | MongoDB | pg_mongo_migration |
| MySQL | PostgreSQL | to_pg_migration |
| SQL Server | PostgreSQL | to_pg_migration |
| Oracle | PostgreSQL | to_pg_migration |
| MongoDB | PostgreSQL | to_pg_migration |

Each engine package follows the same pipeline: **precheck** (no SQL executed) -> **schema** inspection/DDL -> **mapping** (type conversion) -> **engine** (streamed, parameterised inserts so rows are not all held in memory) -> **workflow** (orchestration, progress, cancel) -> verification. The PG-to-MySQL path persists operations in a SQLite store (operation_store.py) using BEGIN IMMEDIATE locks and process-id tracking; interrupted rows are reconciled at start-up. Passwords and connection strings are never stored. backup_proof.py checks that a backup file exists and can be restored into a disposable database - a readable file or a successful dump command is not treated as proof of restorability.

Known limitations (from the matrix): PostgreSQL-to-PostgreSQL copies public tables only; views, functions, triggers, policies, extensions, custom types and non-public schemas are reported but not copied; row-count verification is not a restore test.

# 9. REST API

Base URL `http://localhost:8000/api/v1`; Swagger at /docs, ReDoc at /redoc, OpenAPI at /openapi.json. Errors return `{"detail": "..."}` with 400, 401, 403, 404 or 500.

| Method | Endpoint | Permission | Description |
|---|---|---|---|
| POST | /auth/login | - | Obtain access and refresh tokens |
| GET | /auth/me | token | Current user |
| POST | /backups/ | backup:create | Start a backup |
| GET | /backups/ | backup:read | List backups |
| GET | /backups/{id} | backup:read | Operation status |
| POST | /backups/validate | backup:read | Validate a backup file |
| POST | /backups/retention/apply | backup:create | Apply retention policy |
| POST | /restore/ | restore:create | Restore from backup |
| POST | /patches/execute | patch:create | Execute SQL patch |
| GET | /patches/history | patch:read | Patch history |
| POST | /patches/rollback/{id} | patch:rollback | Rollback patch (admin) |

| Method | Endpoint | Permission | Description |
|---|---|---|---|
| GET | /patches/{id}/report | patch:read | Execution report |
| POST | /fdw/setup \| /extension \| /validate | fdw:manage | FDW management |
| POST | /migrations/preview | migration:create | Preview a migration |
| POST | /migrations/ | migration:create | Run migration |
| POST | /migrations/{id}/cancel \| /reconcile | migration:create | Cancel / reconcile |
| GET | /migrations/{id}/progress | migration:read | Progress |
| GET | /monitoring/health \| /dashboard \| /metrics | monitoring:read | Health, stats, metrics |
| GET | /monitoring/backups/report \| /audit-logs | monitoring:read | Backup report, audit log |
| GET | /monitoring/scheduler/jobs | scheduler:manage | Scheduled jobs |

### 9.1 Role-permission matrix

| Permission | Admin | Operator | Viewer |
|---|---|---|---|
| backup:create / restore:create / patch:create / fdw:manage / migration:create | Yes | Yes | No |
| backup:read / restore:read / patch:read / migration:read / monitoring:read | Yes | Yes | Yes |
| patch:rollback / scheduler:manage / users:manage | Yes | No | No |

# 10. Data Model

Contracts are Pydantic models in `backend/models/schemas.py`.

| Model / enum | Purpose |
|---|---|
| DatabaseType | postgresql, mysql, mongodb, mssql, oracle |
| UserRole | admin, operator, viewer |
| BackupCategory / BackupMode / BackupMethod / BackupSourceType | logical-physical; full-incremental-differential; engine-specific method |
| PgBackupScope, PgDumpFormat, MySQLBackupScope, MySQLDumpFormat | Dump scope and format options |
| MongoRestoreScope, RestoreSourceType, RestoreTargetType | Restore granularity and endpoints |
| OperationStatus | Running / success / failed lifecycle |
| ConnectionRequest, BackupRequest, RestoreRequest, PatchRequest, FDWRequest, MigrationRequest | Request payloads |
| OperationResult, PatchHistoryEntry, HealthCheckResult, DashboardStats | Responses and history |
| UserLogin, UserCreate, UserResponse, TokenResponse | Authentication API |

Persistent runtime data (under var/): saved_servers.json, employees.json, ui_preferences.json, ssh_known_hosts, migration_operations.sqlite, mysql_patch_jobs.json (data/); app.log, audit.log, patch_history.json (logs/); backup output and chain metadata (backups/).

# 11. Desktop Application

CustomTkinter application (`desktop/app.py`, launched by `python -m desktop.main` or `run_app.py`) with a Lucide-icon sidebar (SVG rasterised by resvg_py). Views:

| View | Function |
|---|---|
| Dashboard | KPIs, host gauges (psutil), DB performance chart, infrastructure health, security and activity panels |
| Servers / Databases | Connection profiles, test connect, SSH tunnel; database and object explorer (PostgreSQL, MySQL, Oracle providers) |
| Backup / Restore | Scope, mode and path selection; local or remote artefacts; progress and results |
| Patching | SQL script patch; MySQL and PostgreSQL patch management flows |
| Migration | Source/target selection from the capability matrix, precheck, run, progress |
| Monitoring | Health and performance |
| Reports / Logs | Filterable operations report, execution log with search; CSV and PDF export |
| Settings | Theme (dark/light) and preferences |

Design tokens (palettes, spacing, radius, type scale) live in theme.py; shared widgets in design_system.py and overlay.py (dialogs, toasts). Employee login is handled by employee_store.py and auth_dialog.py.

# 12. Security Design

| Control | Detail |
|---|---|
| Authentication | JWT (HS256) access tokens (60 min dev / 30 min prod) and refresh tokens (7 days); passwords hashed with bcrypt via passlib |
| Authorisation | RBAC with hierarchy admin > operator > viewer; per-endpoint permission strings |
| Secrets | DAP_ environment variables or DAP_SECRETS_FILE; SecretStr; mask_secret() for logs; redaction module for patch audit |
| Credential storage | Windows DPAPI (current-user scope, application entropy) via desktop/windows_dpapi.py |
| SSH | Pinned host keys, mismatch rejected; passwordless sudo only (no sudo prompt); no automatic retry or reboot |
| Data safety | Parameterised SQL in migrations; dry-run/approval gates; operation locks; backup proof by disposable restore |
| Audit | Structured JSON logs; audit.log; patch and migration histories |

## Hardening required before production

- Replace the default demo accounts (admin/admin123, operator/operator123, viewer/viewer123) and move the user store from in-memory to a database or directory service (LDAP/OAuth).
- Set a strong DAP_JWT__SECRET_KEY; the default.yaml value is a development placeholder.
- Restrict CORS origins (currently '*') and terminate TLS at a reverse proxy.
- Encrypt saved connection passwords and stop persisting the login password in ui_preferences.json (noted in deployment/README.md).

- Move writable state out of the install directory (e.g. %LOCALAPPDATA%) for per-machine installs; sign the executable and installer.

# 13. Configuration

Settings are loaded from `config/default.yaml`, overlaid by the profile selected with `DAP_ENVIRONMENT` (dev, test, prod) and then by `DAP_*` environment variables (nested keys use a double underscore) and an optional `.env`.

| Setting | Default | Production override |
|---|---|---|
| environment / debug | dev / true | prod / false |
| api_host / api_port | 0.0.0.0 / 8000 | - |
| connections.default | localhost:5432, db postgres, ssl prefer | set via env |
| backup.output_dir / compression | ./var/backups / gzip | - |
| backup.retention_days | 30 | 90 |
| backup.schedule_cron | 0 2 * * * | 0 1 * * * |
| jwt.access_token_expire_minutes | 60 | 30 |
| jwt.refresh_token_expire_days | 7 | - |
| logging.level / format | INFO / json | WARNING |
| logging paths | ./var/logs/audit.log, app.log | - |

Example environment overrides:

```
DAP_ENVIRONMENT=prod
DAP_JWT__SECRET_KEY=<strong-random-value>
DAP_CONNECTIONS__DEFAULT__HOST=db.example.com
DAP_CONNECTIONS__DEFAULT__PASSWORD=<secret>
DAP_BACKUP__RETENTION_DAYS=90
```

# 14. Installation and Build Guide

## 14.1 Development setup

```
python -m venv venv
venv\Scripts\activate          # Linux/macOS: source venv/bin/activate
pip install -r requirements.txt
copy .env.example .env         # Linux/macOS: cp .env.example .env
```

## 14.2 Running

```
python run_app.py                 # desktop (or: python -m desktop.main)
uvicorn api.main:app --reload     # API; Swagger at http://localhost:8000/docs
python -m http.server 3000 --directory frontend   # optional web dashboard
pytest tests/ -v                  # test suite
```

## 14.3 Production API deployment

- Set DAP_ENVIRONMENT=prod and a real JWT secret; configure credentials via environment or secrets file.
- Run `uvicorn api.main:app --host 0.0.0.0 --port 8000 --workers 4` behind nginx (HTTPS).
- Provision backup disk space and log rotation for var/logs.

### 14.4 Windows installer

Run `deployment\build.bat` (options: exe, installer). Outputs: `dist\Data Automation Pro\Data Automation Pro.exe` and `dist\Data Automation Pro Setup Demo.exe`. The PyInstaller spec uses a flat one-directory layout (contents_directory='.') so existing __file__-relative paths work unchanged. Installation is per-user into %LOCALAPPDATA%\Programs (no UAC); a fixed AppId enables in-place upgrades; user data in var\data, var\logs and var\backups is excluded from the payload and preserved on uninstall. Versions are set only in deployment/version.txt (APP_VERSION, FILE_VERSION).

### 14.5 Troubleshooting

| Symptom | Resolution |
|---|---|
| pg_dump not found | Install PostgreSQL client tools and add to PATH (same for mysqldump, mongodump) |
| psycopg2 not installed | pip install psycopg2-binary |
| Connection refused | Verify host/port, firewall and SSH tunnel settings |
| 401 Unauthorized | Token expired - log in again |
| bcrypt errors | Ensure bcrypt>=4.0.0,<5.0.0 |
| SSH host key rejected | Verify fingerprint on the server; changed keys are never auto-replaced |

## 15. Testing and Quality Assurance

The pytest suite (pytest + pytest-asyncio, httpx for the API) contains 40 modules, run with `pytest tests/ -v`. Coverage areas:

| Area | Test modules |
|---|---|
| API and config | test_api, test_config, test_connectors |
| Backup / restore | test_backup_service, test_backup_modes, test_backup_preview, test_mysql_backup_options, test_restore_gaps, test_mongo_linux_restore, test_mongo_remote_endpoint, test_remote_directory_listing |
| Migration | test_migration_framework, test_pg_migration, test_pg_mysql_migration, test_pg_mysql_api, test_pg_sql_migration, test_pg_mongo_migration, test_to_pg_migration |
| MySQL patching | test_mysql_patch_discovery / precheck / preview / plan / executor / verification / page / remote_backup / centos |
| PostgreSQL patching | test_postgres_patch_admin, _script, _workflow |
| SSH and credentials | test_ssh_host_keys, test_ssh_connection_resolution, test_saved_server_credentials, test_employee_store |
| Dashboard / monitoring | test_monitoring_service, test_dashboard_connection_scope |
| Live / integration (opt-in) | test_pg_migration_integration_live, test_pg_mysql_migration_integration_live, test_postgres_patch_integration_live, test_mysql_patch_integration |

Tests use fake SSH and CLI output and do not connect to servers or install packages; live tests run only when environment variables such as DB_AUTOPRO_MYSQL_PATCH_IT=1 are set. **Continuous integration** (.github/workflows/ci.yml) runs on pushes and pull requests to production, Development, Ganesh and Riya: Windows runner, Python 3.12, install requirements, then `python -m pytest tests/ -q --tb=short`.

# 16. Limitations, Risks and Roadmap

## Current limitations

- Operation tracking for backup/restore is in memory for the API (lost on restart); only migrations are durably stored.
- Desktop roles are largely display-only; RBAC is enforced on the API.
- MySQL patching is dry-run only; Oracle backup via Data Pump is partially wired.
- Migration copies tables and data; views, procedures, triggers and non-public schemas are not migrated.
- Windows installer is unsigned, uses a placeholder icon, and does not package the API or bundle DB client tools.
- Writable state sits beside the binaries; passwords for saved servers need stronger encryption.

## Suggested roadmap

| Phase | Item |
|---|---|
| Short term | Persist operation state (SQLite/PostgreSQL); production user store with password policy; restrict CORS; code signing |
| Medium term | Controlled package installation for MySQL/PostgreSQL patching with rollback guidance; migration of views and routines; additional engines (MariaDB, SQLite, Redis are already recognised in the registry) |
| Long term | Centralised multi-user server mode, alerting integrations, ML-driven anomaly detection using the exported dataset |

# 17. Conclusion

Database Automation Pro delivers a coherent, layered platform that consolidates backup, restore, monitoring, patching, migration and reporting across five database engines. Its emphasis on dry-runs, explicit approvals, pinned SSH trust, durable migration records and an extensive test suite gives it a safety-first character suited to DBA operations. Closing the hardening items listed in sections 12 and 16 will bring it to production readiness.

# Appendix A - Quick reference commands

```
pip install -r requirements.txt
python run_app.py
uvicorn api.main:app --host 0.0.0.0 --port 8000
pytest tests/ -q
deployment\build.bat            # exe + installer
deployment\build.bat exe
```